RESEARCH PROPOSAL AND TECHNICAL GUIDE

Forwarded, Compressed, Misdiagnosed

How messaging-app image channels affect the reliability of image-based diagnostic classifiers

Field: Pattern recognition and image processing

Contents: topic selection, novelty assessment, study design, tech stack, step-by-step implementation, timeline, risks and publication plan

Version: Draft 1, 12 October 2026

> **Read this first.** Novelty claims here come from a limited literature search (about a dozen search rounds), not a full systematic review. Section 4.3 states exactly what was and was not verified, and Section 4.4 gives the searches you must re-run before committing and again before submission.

Contents

1. Executive summary
2. How this topic was selected
3. Background and problem statement
4. Novelty assessment
5. Research questions, hypotheses and objectives
6. Contributions and scope
7. Methodology
8. Technical implementation guide (tech stack and step-by-step instructions)
9. Timeline
10. Risks and mitigations
11. Ethics, terms of service and data handling
12. Publication plan
13. Backup plan
14. References

1. Executive summary

Images used for disease diagnosis often reach a classifier through a messaging app such as WhatsApp or Telegram. These apps resize and recompress photos. This study asks whether that hidden step changes a classifier's predictions, and, more importantly, whether the classifier's confidence warns anyone when it does.

The central risk is silent failure: a prediction that flips to a wrong class while confidence stays high. A recent study on generic image classifiers found that lossy compression shifts predictions without substantially eroding confidence [8]. Some deployed advisory systems rely on low confidence to ask the user for a better photo [14]. If compression flips predictions without lowering confidence, that safeguard does not trigger.

Item	Summary
Working title	Forwarded, Compressed, Misdiagnosed: how messaging-app image channels affect the reliability of image-based diagnostic classifiers
Core contribution	First systematic measurement (as far as the searches found) of how real messaging channels change accuracy, calibration and confident-error rates of disease classifiers, plus a validated channel emulator and a comparison of mitigations
Novelty confidence	Medium. Nothing found that studies this exact question for disease classifiers; the individual methods are borrowed from image forensics. See Section 4
Paper type	Empirical study with a measurement-based emulator and a mitigation comparison. Not a new architecture
Realistic venues	Mid-tier open-access and applied journals (Section 12). Not a top-tier methods venue
Compute needs	Light. Free Colab or Kaggle GPU is enough for training small models; most analysis runs on stored predictions
Timeline	About 16 weeks part-time (Section 9)
Biggest risk	The effect may be small because models downscale inputs to about 224 pixels, which can mask recompression. A one-week pilot decides this (Section 8.5, Step 1)

Item	Summary
First action	Run the pilot: about 300 images through WhatsApp and Telegram, then compare predictions against the originals

2. How this topic was selected

Several directions were screened against the literature before settling on this one. Most failed because they were already crowded or lacked usable data.

Candidate topic	Verdict	Reason
Maize reliability under distribution shift (original draft)	Crowded	A 2026 benchmark already studies calibration, selective prediction and Grad-CAM under lab-to-field shift [1]; related work covers uncertainty-aware cross-domain evaluation [2] and maize calibration [3]
Conformal prediction for plant disease	Crowded	Already published as a full framework [4]
Quality-aware calibration	Taken in forensics	Done for AI-generated image detection [5]; not found for disease classifiers
SAR target recognition with uncertainty	Crowded, excluded	Heavily covered, and too close to targeting applications to pursue
Landmine detection with conformal risk control	Runner-up	Method is established and datasets exist [19][20]; novelty is application only, with an active competitor
Tobacco leaf grading	Saturated	Reported accuracies near 99.9% on large datasets [23]
Mining and ore identification	Covered	Satellite mapping, mineral apps and ore sorting exist [26]
Livestock skin disease	Crowded	Many near-identical papers on one public dataset [24]
Skin-related neglected tropical diseases	Data-limited	Under-studied [25] but data are not openly available
Rock-art layer ordering	Backup	Strongest novelty, but needs expert labels and a collaborator (Section 13) [21][22]
Messaging-channel reliability (this proposal)	**Selected**	No direct prior work found; cheap pilot; reuses a standard classification pipeline

3. Background and problem statement

Deep classifiers for plant and skin disease report very high accuracy on clean benchmark images. Deployment is different. In low-connectivity settings, photos frequently travel through messaging apps before they are analysed, for example to a WhatsApp-based advisory bot. Such bots exist and are described mainly in system terms [12][13].

Platforms do alter images. One study of the Facebook pipeline found that it resizes and recompresses uploads depending on their size and quality [9]. Image-forensics research has studied this extensively, including real transmission through Facebook, WeChat, Weibo and WhatsApp for forgery localization [11] and platform chains for AI-generated image detection [6][10]. One group

even estimated platform compression parameters from a small number of uploads and built a local emulator [7].

In contrast, robustness work on crop and plant disease classifiers mostly applies synthetic corruptions at fixed levels, such as blur, noise, brightness and JPEG quality [15][16][17]. Those tests do not reproduce what a real platform does, and they do not examine multi-step forwarding.

Problem statement. It is unknown whether real messaging-app image channels degrade disease classifiers, whether the degradation is visible in the classifier's confidence, and which low-cost mitigations work. Without this, systems that depend on confidence to decide when to ask for a retake or refer a case may fail silently.

4. Novelty assessment

4.1 What already exists

Area	Representative work	Why it does not close the gap
Lab-to-field reliability for plant disease	[1] [2]	Models shift between datasets, not through an image channel
Maize calibration and Grad-CAM	[3]	Calibration measured on clean test data
Conformal and uncertainty methods in plant disease	[4]	Focus on model design, not on input transmission
Synthetic corruption benchmarks in crops	[15] [16] [17]	Fixed-level synthetic JPEG, blur and noise. One adds calibration [17] but is still synthetic
Quality-aware calibration	[5]	AI-generated image detection, not disease classification
Real platform chains	[6] [10] [11]	Forensics tasks. Shows the evaluation method is established
Platform compression emulation	[7]	Video deepfake detection. Transferable method
Compression versus classifier confidence	[8]	Generic ImageNet classifiers. Indicates predictions shift without a confidence drop
WhatsApp advisory bots	[12] [13] [14]	System descriptions reporting clean accuracy. One uses low confidence to request re-upload [14]

4.2 The gap

No study found that (a) measures real messaging-channel effects on disease classifiers, (b) includes multi-hop forwarding and a "send as file" control, (c) validates an emulator by decision agreement with real channels, and (d) tests channel-aware mitigations including provenance-conditioned calibration against a quality-aware baseline.

4.3 Confidence and limitations

- **Confidence is medium, not high.** About a dozen search rounds were run with a general-purpose search engine. This is not a systematic review of Scopus or Web of Science.

- **Absence of search results is not proof of absence.** Related work may exist under different terminology, such as "image sharing platforms", "OSN transmission" or "telemedicine image quality".
- **The novelty is in the setting and the evaluation, not the algorithms.** Emulation, quality-aware calibration and platform-chain benchmarks are borrowed from forensics. Reviewers may call this incremental, so the paper must lead with measurement rigour and practical findings.
- **Platforms change silently.** Results are tied to app versions and dates, which must be recorded.
- **Competing preprints are possible.** The field moves quickly; re-check before submission.

4.4 Re-check protocol

Run these on Scopus, Web of Science and Google Scholar. Log the date, database, query and number of relevant hits. Repeat before submission.

```
"WhatsApp" OR "messaging app" AND "plant disease" OR "skin lesion" AND "compression"
"social network" AND "recompression" AND "image classification" AND "reliability"
"forwarding" OR "multi-hop" AND "JPEG" AND "classification" AND "confidence"
"compression history" AND "confidence calibration"
"instant messaging" AND "image quality" AND "telemedicine" AND "deep learning"
"online social network" AND "robustness" AND "disease classification"
```

5. Research questions, hypotheses and objectives

5.1 Research questions

ID	Question
RQ1	How much do real messaging channels change the predictions, accuracy and calibration of disease classifiers?
RQ2	Does multi-hop forwarding compound the degradation?
RQ3	Does sending images "as file/document" remove most of the harm?
RQ4	Can an emulator fitted from a small number of real uploads reproduce real-channel decisions?
RQ5	Do channel-aware mitigations (augmentation and provenance-conditioned calibration) reduce confident errors, compared with a quality-aware baseline?

5.2 Hypotheses (each must be falsifiable)

ID	Hypothesis	How it could be refuted
H1	Real channels produce a prediction flip rate above the near-lossless control (document mode)	Flip rates are indistinguishable from the control
H2	Confidence does not drop in proportion to accuracy, so the confident-error rate rises	Confidence falls as errors rise, and calibration is unchanged
H3	Additional forwarding hops add measurable degradation only if the platform re-encodes forwarded media	Hops two to four are identical to hop one

ID	Hypothesis	How it could be refuted
H4	An emulator fitted from a small probe set reaches high decision agreement with real channels (target to be set after the pilot; for example 98% or higher)	Agreement stays low even after tuning resize and quantization parameters
H5	Channel-aware augmentation plus provenance-conditioned calibration lowers the confident-error rate versus a clean-calibrated baseline	No improvement, or no improvement over the quality-aware baseline

> **Treat negative results as results.** If the pilot shows a tiny effect, the paper becomes a measurement study showing that common channels are mostly benign and identifying exactly where they are not. That is still publishable and useful.

5.3 Objectives

1. Build a clean, audited, leakage-free training and test split for a maize disease dataset and one second domain.
2. Train lightweight baseline classifiers and record per-image logits.
3. Measure real channel outputs (WhatsApp, Telegram and optionally Messenger) in photo and file modes, up to four hops.
4. Extract channel fingerprints: resize rule, JPEG quantization tables, chroma subsampling, metadata handling.
5. Build and validate an emulator by pixel similarity and by decision agreement.
6. Quantify degradation using flip rate, accuracy, calibration and confident-error metrics.
7. Evaluate mitigations: channel-aware augmentation, temperature scaling variants, provenance-conditioned calibration and a no-reference quality baseline.
8. Release code, splits, emulator parameters and measurement logs.

6. Contributions and scope

- **Measurement:** a dated, versioned characterization of real messaging-channel transformations.
- **Emulator:** a small open-source emulator validated by decision agreement, not only pixel similarity.
- **Reliability analysis:** effects on accuracy, calibration, confident errors and selective prediction, including multi-hop forwarding and a file-mode control.
- **Mitigation comparison:** channel-aware augmentation and provenance-conditioned calibration versus a quality-aware baseline.
- **Practical guidance:** concrete advice for systems that receive images through messaging apps.

Out of scope and not claimed: solving agricultural or clinical domain shift in general; replacing expert diagnosis; results for platforms or app versions that were not measured.

7. Methodology

7.1 Study design in one view

1. Take clean test images from audited datasets.

2. Pass the same images through real channels (and record every output file).
3. Fit an emulator from the measurements and validate it.
4. Run trained classifiers on clean, real-channel, emulated-channel and synthetic-JPEG versions.
5. Compare predictions and confidence across conditions using the metrics in 7.6.
6. Test mitigations on real channels, never only on the emulator that trained them.

7.2 Datasets

Dataset	Role	Notes
PlantVillage maize classes (healthy, common rust, northern leaf blight, gray leaf spot)	Primary training and test	Audit for near-duplicates and background bias; split by duplicate groups, not by image
Independent field maize dataset (candidate; not verified here)	External domain test	Verify licence, label quality and class overlap before use
Phone-captured skin lesion dataset such as PAD-UFES-20 (candidate; not verified here)	Second domain for generality	Verify licence and usage terms. Any open, licensed, phone-captured diagnostic set can replace it
Probe set (20 to 30 images at several resolutions)	Channel characterization	Your own photos or public-domain images at about 640, 1280, 2048 and 4000 pixels on the long side

7.3 Models

- **Model A (baseline):** MobileNetV3-Large and ResNet-18 (or EfficientNet-B0), fine-tuned on clean images.
- **Model B (channel-augmented):** same architectures trained with random resize and JPEG chains drawn from measured parameters.
- **Resolution ablation:** evaluate at 224 pixels and at a larger input size (for example 384), because downscaling can hide recompression.
- **Optional Model C:** a five-member deep ensemble for an uncertainty reference.

7.4 Channels

Channel	Mode	Hops	Purpose
WhatsApp (Android)	Photo	1 to 4	Primary real channel
WhatsApp	Document/file	1	Near-lossless control
Telegram	Compressed photo	1 to 4	Second real channel
Telegram	File	1	Near-lossless control
Messenger (optional)	Photo	1	Third platform if time allows
Synthetic JPEG ladder	Quality 50 to 95	1	What standard corruption benchmarks do; compared against real channels
Fitted emulators	Per channel	n	Augmentation and large-scale evaluation

7.5 Reliability and calibration variants

ID	Variant	Description
C0	Raw softmax	Maximum softmax probability as confidence
C1	Temperature scaling (clean)	Temperature fitted on clean validation images
C2	Temperature scaling (channel-aware)	Temperature fitted on emulated-channel validation images
C3	Provenance-conditioned	Small model predicting correctness from softmax features plus file fingerprint (dimensions, bytes per pixel, quantization level, subsampling, metadata presence)
C4	Quality-aware baseline	Same idea using a no-reference image quality score, following [5]
C5	Deep ensemble (optional)	Reference uncertainty method

7.6 Metrics

Metric	Why it matters
Accuracy and macro-F1	Standard classification performance
Prediction flip rate versus the original image	Label-free measure of what the channel changes; the core metric
ECE (15 bins), Brier score, NLL	Calibration quality
Confident-error rate (confidence at least 0.9 and wrong)	The silent-failure measure at the heart of the study
Failure-prediction AUROC and AURC	How well confidence ranks correct above incorrect
Selective risk at fixed coverage (90% and 80%)	Value of rejecting uncertain cases
Emulator fidelity: PSNR, SSIM, decision agreement	Does the emulator behave like the real channel?

7.7 Experimental matrix

Datasets (2) by architectures (2) by seeds (5) by conditions (about 8 to 10) by calibration variants (5). Calibration variants and most metrics run post hoc on stored logits, so only the model training runs are expensive. Compare Model A and Model B in the main ablation, then add calibration variants on top.

7.8 Statistical analysis

- Bootstrap 95% confidence intervals over test images (resampling images, not predictions).
- Paired McNemar test for flips between conditions; paired Wilcoxon tests across seeds for model comparisons.
- Holm correction for multiple comparisons; report effect sizes, not only p-values.
- Report the control (document mode) next to every real-channel result.

8. Technical implementation guide

This section covers what to install, how to organize the project and how to carry out each step. Command-line paths and package versions can change; verify them on your machine and pin what you use.

8.1 Tech stack

Layer	Tool	Used for
Language	Python 3.11 or later	Everything
Deep learning	PyTorch, torchvision, timm	Pretrained MobileNetV3, ResNet-18 and EfficientNet-B0; training and inference
Image handling	Pillow, OpenCV, imagehash, piexif	Encoding with chosen JPEG settings, reading quantization tables, perceptual-hash matching, metadata
Image quality	pyiqa (IQA-PyTorch), scikit-image	No-reference quality scores (BRISQUE, NIQE, MUSIQ), PSNR and SSIM
Calibration	netcal or your own functions, scikit-learn	ECE, temperature scaling, logistic or boosted correctness predictor
Statistics	NumPy, SciPy, statsmodels, pandas	Bootstrap, McNemar, Wilcoxon, Holm correction
Explainability (optional)	pytorch-grad-cam	Grad-CAM for flipped predictions
Phone tooling	Android Debug Bridge (adb), two Android phones	Pulling received files from WhatsApp and Telegram
Config and tracking	Hydra or YAML, Weights & Biases or MLflow, Git	Reproducible experiments
Data and artifacts	DVC or Git LFS, Parquet files	Versioning datasets and per-image logits
Edge check (optional)	ONNX and ONNX Runtime	Latency of the lightweight models
Plots and writing	Matplotlib, Seaborn, Overleaf (LaTeX), Zotero	Figures, manuscript, references
Compute	Google Colab or Kaggle notebooks; laptop CPU for analysis	Free GPU is enough for small models

8.2 Hardware and accounts

- Two Android phones (or one phone plus a second account on another device), each with WhatsApp and Telegram installed. Record the OS version and app version for every measurement session.
- A USB cable and developer options enabled so adb can read the phones.
- A free Colab or Kaggle account for GPU training; a local machine with 16 GB RAM is comfortable for analysis.
- Accounts should be your own. Do not automate sending (see Section 11).

8.3 Environment setup

```
python -m venv .venv
source .venv/bin/activate        # Windows (Git Bash): source .venv/Scripts/activate
pip install torch torchvision timm pillow opencv-python numpy pandas scipy
pip install scikit-learn scikit-image statsmodels imagehash piexif pyarrow
pip install netcal pyiqa pytorch-grad-cam hydra-core wandb matplotlib seaborn tqdm
pip freeze > requirements.txt    # pin exact versions for the paper
```

8.4 Repository structure

```
channel-study/
  configs/                 # YAML: dataset paths, model, seeds, channel params
  data/
    raw/                   # original datasets (never edited)
    splits/                # CSVs of group-aware train/val/test splits
    channels/              # received files: wa_photo_hop1/, tg_file/, ...
    channel_log.csv        # app, version, mode, hop, original id, fingerprint
  src/
    audit.py               # duplicate grouping and split creation
    train.py               # fine-tune models, save checkpoints
    infer.py               # run models on any folder, write logits parquet
    fingerprint.py         # JPEG fingerprint and quality estimation
    emulate.py             # channel emulator and augmentation transform
    metrics.py             # ECE, flip rate, confident-error rate, risk-coverage
    calibrate.py           # temperature scaling and provenance-conditioned model
    stats.py               # bootstrap, McNemar, Wilcoxon, Holm
  results/                 # logits parquet, tables, figures
  notebooks/               # exploration only
  README.md
```

8.5 Step-by-step instructions

Step 1. Run the pilot (week 1)

Goal: find out quickly whether real channels change predictions at all.

1. Download the PlantVillage maize classes. Hold out about 300 images across the four classes as a pilot test set.
2. Fine-tune MobileNetV3 on the rest for a few epochs (Step 3 code). Any reasonable accuracy is enough for the pilot.
3. Send the 300 images from phone A to phone B via WhatsApp as photos (in batches), then again as documents. Repeat with Telegram as compressed photos and as files.
4. Pull the received files, match them to the originals (Step 4), and run the model on originals and received copies.
5. Compute flip rate, ECE change and confident-error rate for each channel (Step 7 functions).

Pilot outcome (suggested thresholds)	Decision
Flip rate of about 1 to 2% or more on any photo-mode channel, or a clear rise in confident errors	Proceed with the full study
Flip rate near zero at 224 pixels	Re-run at higher input resolution and on small-lesion images; test multi-hop; if still nothing, write a short negative-result paper or move to the backup topic
Flip rate high only for photo mode and near zero for file mode	Strong story: the free "send as file" mitigation matters. Proceed

These thresholds are suggestions. Agree the decision rule with your supervisor before looking at the results.

Step 2. Audit the dataset and build leakage-free splits

PlantVillage contains near-duplicate images of the same leaf. If duplicates sit in both training and test sets, accuracy is inflated. Cluster near-duplicates with perceptual hashes and split by cluster.

```
import numpy as np, imagehash
from PIL import Image
from scipy.spatial.distance import cdist
from scipy.sparse import csr_matrix
from scipy.sparse.csgraph import connected_components
from sklearn.model_selection import GroupShuffleSplit

H = np.array([imagehash.phash(Image.open(p).convert("RGB")).hash.flatten()
              for p in paths], dtype=np.uint8)
dist = cdist(H, H, "hamming") * 64            # Hamming distance in bits
n_groups, group = connected_components(csr_matrix(dist <= 6))

gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=0)
train_idx, test_idx = next(gss.split(paths, labels, groups=group))
```

The threshold of 6 bits is a starting point; inspect a sample of grouped images by eye. For more than about 10,000 images, compute distances in chunks. Repeat the split to get validation data, and save the final splits as CSV files.

Step 3. Train baselines

```
import timm, torch, torch.nn as nn

model = timm.create_model("mobilenetv3_large_100", pretrained=True,
num_classes=4).cuda()
opt = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=1e-2)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=15)

for epoch in range(15):
    model.train()
    for x, y in train_loader:
        x, y = x.cuda(), y.cuda()
        loss = nn.functional.cross_entropy(model(x), y)
        opt.zero_grad(); loss.backward(); opt.step()
    sched.step()
```

- Use one fixed evaluation preprocessing pipeline (resize method, crop, normalization) and record it. The channel resizes the image first, then your pipeline resizes again; the interaction matters.
- Train 5 seeds per configuration. Save checkpoints and the validation set predictions.
- Write per-image logits to Parquet from infer.py for every condition. All later analysis reads these files.

Step 4. Measure the real channels

Protocol for each app, mode and hop:

1. Create a fixed ordering of the probe and test images. Send them from phone A to phone B in small batches (apps limit how many photos go in one message; check on your version).

2. For file or document mode, use the app's "send as document/file" option for the same images.
3. For hops 2 to 4, forward the received images onward (for example B to a second chat, then to another account) and collect the final copies after each hop.
4. Pull the files from phone B with adb or a file manager. Storage paths vary by Android version and app version, so find the right folder first.
5. Log app version, OS version, date, device model and network type (Wi-Fi or mobile data) in channel_log.csv.
6. Repeat the whole measurement on a second device or OS version, and again after any major app update.

```
adb devices
# Example only: verify the real folder on your phone first
adb pull "/sdcard/Android/media/com.whatsapp/WhatsApp/Media/WhatsApp Images/" \
    data/channels/wa_photo_hop1/
```

Received files are renamed by the app and the order is not guaranteed, so match them to the originals by perceptual hash:

```
import imagehash, glob
from PIL import Image

def phash(p): return imagehash.phash(Image.open(p).convert("RGB"))

orig = {p: phash(p) for p in glob.glob("data/raw/pilot/*.jpg")}
recv = {p: phash(p) for p in glob.glob("data/channels/wa_photo_hop1/*.jpg")}

pairs = {}
for rp, rh in recv.items():
    best = min(orig, key=lambda op: orig[op] - rh)   # Hamming distance
    pairs[rp] = (best, orig[best] - rh)
# Flag matches with distance > 8 or two receivers mapped to one original
```

Extract the channel fingerprint from every received JPEG:

```
import os
from PIL import Image, JpegImagePlugin

def jpeg_fingerprint(path):
    im = Image.open(path)
    q = getattr(im, "quantization", None)        # dict: table id -> 64 values
    size = os.path.getsize(path)
    return {
        "file": os.path.basename(path),
        "width": im.width, "height": im.height,
        "bytes": size, "bpp": size * 8 / (im.width * im.height),
        "q_lum": list(q[0]) if q else None,
        "q_chr": list(q[1]) if q and 1 in q else None,
        "subsampling": JpegImagePlugin.get_sampling(im),   # 0=4:4:4, 1=4:2:2,
2=4:2:0
        "has_exif": bool(im.getexif()),
    }
```

To summarize a quantization table as a "quality" number, compare it against tables Pillow produces at each quality. An exact match (error zero) means standard libjpeg scaling; a non-zero error means a custom table, which you should store and reuse directly.

```
import io
```

```
from PIL import Image

def estimate_quality(path):
    target = Image.open(path).quantization[0]
    probe = Image.new("RGB", (64, 64))
    best, best_err = None, 1e9
    for q in range(30, 101):
        buf = io.BytesIO(); probe.save(buf, "JPEG", quality=q); buf.seek(0)
        cand = Image.open(buf).quantization[0]
        err = sum(abs(a - b) for a, b in zip(target, cand))
        if err < best_err: best, best_err = q, err
    return best, best_err
```

Also characterize the resize rule: send the probe images at 640, 1280, 2048 and 4000 pixels and tabulate input versus output dimensions. Look for a maximum long side, an aspect-ratio rule and whether small images are left alone.

Step 5. Build the emulator

```
import io
from PIL import Image, ImageOps

def emulate_channel(img, params):
    img = ImageOps.exif_transpose(img).convert("RGB")
    w, h = img.size
    scale = min(1.0, params["max_side"] / max(w, h))
    if scale < 1.0:
        img = img.resize((round(w * scale), round(h * scale)), params["resample"])
    buf = io.BytesIO()
    img.save(buf, "JPEG", qtables=params["qtables"],
subsampling=params["subsampling"])
    buf.seek(0)
    return Image.open(buf).convert("RGB")      # EXIF is dropped automatically
```

Choose the resampling filter by trying each one and keeping the lowest pixel error against real received images:

```
import numpy as np
from PIL import Image

RES = {"bilinear": Image.BILINEAR, "bicubic": Image.BICUBIC, "lanczos":
Image.LANCZOS}

def best_resampler(orig, received, params):
    errs = {}
    for name, rs in RES.items():
        out = emulate_channel(orig, {**params, "resample": rs}).resize(received.size)
        diff = np.asarray(out, float) - np.asarray(received.convert("RGB"), float)
        errs[name] = float(np.mean(diff ** 2))
    return min(errs, key=errs.get), errs
```

Fit parameters on part of the probe and test images, and keep another part for validation (Step 6). Store all parameters in a YAML file with the app version and date.

Step 6. Validate the emulator

- **Pixel fidelity:** PSNR and SSIM between emulated and real received images (skimage.metrics).
- **Decision agreement (the important one):** the fraction of images where the model predicts the same class on the real and the emulated copy, plus the difference in confidence.

- **Held-out check:** evaluate on images and on a device or app version not used for fitting.
- **Compare with plain JPEG:** show that a simple "JPEG at quality 70" corruption agrees with the real channel less well than the fitted emulator.

Step 7. Run the reliability experiments

Run each model on every condition (clean, each real channel and hop, document-mode control, fitted emulators, synthetic JPEG ladder) and save logits. Then compute metrics from the logits:

```
import numpy as np

def ece(conf, correct, n_bins=15):
    edges, total = np.linspace(0, 1, n_bins + 1), 0.0
    for lo, hi in zip(edges[:-1], edges[1:]):
        m = (conf > lo) & (conf <= hi)
        if m.any():
            total += m.mean() * abs(correct[m].mean() - conf[m].mean())
    return total

def confident_error_rate(conf, correct, thr=0.9):
    return float(np.mean((conf >= thr) & (~correct)))

def flip_rate(pred_original, pred_channel):
    return float(np.mean(pred_original != pred_channel))

def risk_coverage(conf, correct):
    order = np.argsort(-conf)
    err = (~correct[order]).astype(float)
    n = len(err)
    risk = np.cumsum(err) / np.arange(1, n + 1)
    coverage = np.arange(1, n + 1) / n
    aurc = float(np.sum(np.diff(coverage, prepend=0) * risk))
    return coverage, risk, aurc
```

Step 8. Test mitigations

8a. Channel-aware augmentation. Add a transform that applies one to three emulated hops with parameters drawn from the measured channels:

```
import random

class ChannelAug:
    def __init__(self, channels, p=0.7):
        self.channels, self.p = channels, p
    def __call__(self, img):
        if random.random() > self.p:
            return img
        for _ in range(random.choice([1, 1, 2, 3])):     # simulates forwarding
            img = emulate_channel(img, random.choice(self.channels))
        return img
```

8b. Temperature scaling. Fit the temperature on clean validation images (C1) and on emulated-channel validation images (C2):

```
import torch, torch.nn.functional as F

def fit_temperature(logits, labels):
    T = torch.nn.Parameter(torch.ones(1))
    opt = torch.optim.LBFGS([T], lr=0.1, max_iter=100)
```

```
    def closure():
        opt.zero_grad()
        loss = F.cross_entropy(logits / T, labels)
        loss.backward()
        return loss
    opt.step(closure)
    return T.item()
```

8c. Provenance-conditioned calibration (C3). Train a small model to predict whether a prediction is correct, using softmax features plus the file fingerprint. Fit it on calibration data that include emulated channels, then test on real channels.

```
import numpy as np
from sklearn.linear_model import LogisticRegression

X = np.c_[max_prob, entropy, np.log(bpp), min_side, q_lum_mean, subsampling,
has_exif]
clf = LogisticRegression(max_iter=1000).fit(X_cal, correct_cal)
p_correct = clf.predict_proba(X_test)[:, 1]    # new confidence score
```

8d. Quality-aware baseline (C4). Compute a no-reference quality score for every image and use it in place of the fingerprint features:

```
import pyiqa
iqa = pyiqa.create_metric("brisque")      # or "niqe", "musiq"
score = float(iqa("path/to/image.jpg"))
```

Compare C0 to C4 using failure-prediction AUROC, AURC, confident-error rate and selective risk. Always evaluate on real channels that were not used to fit the emulator, and keep the fitting data separate from the test data.

Step 9. Statistics and figures

```
import numpy as np
from statsmodels.stats.contingency_tables import mcnemar
from statsmodels.stats.multitest import multipletests

rng = np.random.default_rng(0)
def bootstrap_ci(metric_fn, y, p, n=2000):
    idx = rng.integers(0, len(y), (n, len(y)))
    vals = [metric_fn(y[i], p[i]) for i in idx]
    return np.percentile(vals, [2.5, 97.5])

# paired comparison of two conditions on the same images
table = [[both_correct, a_only], [b_only, both_wrong]]
p = mcnemar(table, exact=True).pvalue
# after collecting all p-values:
reject, p_adj, _, _ = multipletests(p_values, method="holm")
```

- Main figures: flip rate by channel and hop; reliability diagrams; risk-coverage curves; confident-error rate across mitigations; emulator versus real agreement.
- Always plot the document-mode control next to the photo-mode result.
- Inspect Grad-CAM on a handful of flipped cases to see whether the model attends to different regions after compression.

Step 10. Reproducibility package

- Pin dependencies (requirements.txt), seeds and configs; log runs with Weights & Biases or MLflow.

- Release splits, the channel log, the fitted emulator parameters with app versions and dates, and the logits files.
- Publish code on GitHub and archive a release on Zenodo to get a DOI.
- Write a README that lets someone reproduce each table with one command.

8.6 Compute and timing

Fine-tuning small models on a few thousand images takes minutes to tens of minutes per run on a free GPU; exact times depend on the hardware. Expect a handful of GPU hours in total for 5 seeds by 2 architectures by 2 datasets, plus data loading overhead. The channel measurements are manual and take a few hours per app. Everything after inference is CPU analysis.

8.7 Common pitfalls

- **Duplicate leakage** between training and test sets inflates baseline accuracy. Group before splitting.
- **Wrong matching** of received files to originals. Use hashes and flag poor matches.
- **Orientation and EXIF:** apply the same orientation handling everywhere or you will measure rotation, not compression.
- **Unrecorded preprocessing.** Fix and report the evaluation resize and crop.
- **Testing mitigations on the emulator that trained them.** Evaluate on real channels.
- **Reporting only accuracy.** Include flip rate and confident-error rate.
- **Threshold cherry-picking.** Fix thresholds on validation data before looking at test results.
- **App version drift.** Re-measure after updates and record versions.

9. Timeline

Weeks	Work	Output
1	Pilot (Step 1) and go/no-go decision	Short pilot note and decision
2 to 3	Dataset audit, splits, baselines (Steps 2 and 3)	Splits, trained models, logits
4 to 5	Real-channel measurements (Step 4)	Channel log, fingerprints, resize and quality tables
6	Emulator build and validation (Steps 5 and 6)	Emulator, agreement results
7 to 9	Main reliability experiments (Step 7), second domain	Metrics tables and plots
10 to 11	Mitigations (Step 8)	Augmentation and calibration comparison
12	Statistics, figures, Grad-CAM cases (Step 9)	Final figures and tables
13 to 15	Writing; refresh literature search	Full draft
16	Supervisor review, reproducibility package, submission	Submitted manuscript and public repository

10. Risks and mitigations

Risk	Likelihood	Mitigation
Effect is tiny at 224 pixels	Medium	Pilot early; add higher-resolution inputs and small-lesion data; multi-hop; fall back to a negative-result framing
Reviewers call it incremental	Medium to high	Lead with measurement rigour, emulator validation by decision agreement, and the practical file-mode finding
App behaviour changes during the study	Medium	Record versions and dates; re-measure; release emulator parameters per version
Competing preprint appears	Low to medium	Re-run searches monthly; post a preprint early
Second dataset licence or ethics problem	Medium	Verify terms first; swap for any open licensed phone-captured set
File matching errors	Low	Perceptual hashes with manual spot checks
Terms-of-service issues with messaging apps	Low if manual	Own accounts, manual sending, no scraping or automation
Overclaiming clinical or agronomic value	Medium	State clearly that this is a reliability study, not a diagnostic tool

11. Ethics, terms of service and data handling

- **Terms of service:** send images manually from your own accounts. Do not automate, scrape or use unofficial clients.
- **Data:** use public datasets with clear licences. Do not send private patient images through messaging apps for this study.
- **Institution:** discuss the plan with your supervisor and check whether your university requires ethics review, especially for the skin-lesion domain.
- **Heritage of results:** report the app versions and dates; platform behaviour is not permanent.
- **Claims:** do not present outputs as medical or agronomic advice.
- **Disclosure:** follow the target journal's policy on AI-tool use in writing and analysis.

12. Publication plan

12.1 Realistic venues

These are possible fits to check against current scope, article charges and acceptance patterns before choosing; I have not verified their current requirements.

- Frontiers in Plant Science, Scientific Reports, IEEE Access, Smart Agricultural Technology, Plants or Agronomy for an agricultural framing.
- Pattern Recognition Letters, IET Image Processing or Journal of Imaging if the second domain makes the paper more general.
- Top-tier methods venues are unlikely because the contribution is empirical.

12.2 Paper outline

1. Introduction: messaging channels, silent failure, gap, contributions.
2. Related work: plant and medical image reliability; synthetic corruption benchmarks; forensics platform chains; calibration and selective prediction.
3. Methods: datasets and audit; channel measurement protocol; emulator; models; mitigations; metrics; statistics.
4. Results: channel fingerprints; flip rate and accuracy; calibration and confident errors; multi-hop; document-mode control; emulator validation; mitigations; second domain.
5. Discussion: practical guidance, failure cases, platform drift.
6. Limitations and conclusion.

12.3 Submission checklist

- Re-run the Section 4.4 searches and update the related-work section.
- Post a preprint if the journal allows it.
- Prepare data and code availability statements and a Zenodo DOI.
- Write a cover letter that states the gap and the practical finding in two sentences.
- Suggest reviewers outside your own group; prepare a response-to-reviewers template.

13. Backup plan

- **If the pilot shows no effect:** re-test at higher resolution, with multi-hop and with small-lesion field images. If it is still null, write a short measurement paper showing where channels are benign.
- **If you gain an archaeologist collaborator and access to real panels:** consider the rock-art layer-ordering topic, which has the strongest novelty but needs expert labels [21][22].
- **If you want a different angle on the same data:** the landmine conformal risk-control topic reuses the same reliability toolkit [19][20].

14. References

These are the sources consulted during topic screening. Verify each before citing.

[1] Quantifying the reliability gap in cross-domain plant disease classification (Frontiers in Plant Science, 2026). https://www.frontiersin.org/journals/plant-science/articles/10.3389/fpls.2026.1826962/full

[2] An uncertainty-aware evaluation framework based on hierarchical vision transformers for robust cross-domain plant leaf disease classification (Scientific Reports, 2026). https://www.nature.com/articles/s41598-026-55107-6

[3] Enhanced Maize Leaf Disease Detection and Classification Using an Integrated CNN-ViT Model. https://pmc.ncbi.nlm.nih.gov/articles/PMC12208913/

[4] MRC-Net: a reliable plant disease classification framework with multi-frequency state-space enhancement and conformal prediction (Frontiers in Plant Science, 2026). https://www.ncbi.nlm.nih.gov/pmc/articles/PMC13582607/

[5] Quality-Aware Calibration for AI-Generated Image Detection in the Wild (QuAD). https://arxiv.org/pdf/2604.15027

[6] GlobalForge: Towards Robust AI-Generated Image Detection (RealDeg-Bench). https://arxiv.org/pdf/2607.14684

[7] Bridging the Gap: A Framework for Real-World Video Deepfake Detection via Social Network Compression Emulation. https://arxiv.org/pdf/2508.08765

[8] Discrete Atomic Compression of JPEG Images: evaluation across CNN classifiers (Signals, 2026). https://doi.org/10.3390/signals7050092

[9] Robust image steganography against lossy JPEG compression (Facebook recompression observations). https://arxiv.org/pdf/2304.13297

[10] Bridging the Gap Between Ideal and Real-world Evaluation: Benchmarking AI-generated image detection (RRBench). https://arxiv.org/pdf/2509.09172

[11] SDiFL: Stable Diffusion-Driven Framework for Image Forgery Localization (robustness on online social networks). https://arxiv.org/pdf/2508.20182

[12] KisanAI: Flask-based WhatsApp chatbot for farmers (GitHub). https://github.com/mandarwagh9/KisanAI

[13] AgriBot: an intelligent chatbot for farmers with crop and disease prediction. https://www.researchgate.net/publication/399563618

[14] Integrated Agricultural Advisor: AI chatbot for crop disease and recommendation. https://www.ijtas.com/papers/2026/4/1253.pdf

[15] From Code to Field: Evaluating the Robustness of CNNs for Disease Diagnosis in Mango Leaves. https://arxiv.org/pdf/2512.13641

[16] AgroVisNet and the BD-PlantDX benchmark (corruption robustness). https://arxiv.org/pdf/2609.10469

[17] Rice disease diagnosis with calibration and corruption-based robustness analysis. https://doras.dcu.ie/32440/1/1-s2.0-S2214317326000259-main.pdf

[18] Empirically Validating Conformal Prediction on Modern Vision Architectures Under Distribution Shift. https://arxiv.org/pdf/2307.01088

[19] SULAND_v2: surface landmine detection dataset under domain shift. https://arxiv.org/html/2607.28996

[20] AMLID: Adaptive Multispectral Landmine Identification Dataset. https://arxiv.org/pdf/2512.18738

[21] Hierarchical and related machine-learning work on UXO thermal imagery (UXOTi_NPA). https://www.mdpi.com/2072-4292/15/4/967

[22] A Multi-Stage Deep Learning Framework for Automatic Pigment Classification and Filter Prediction in Rock Art Paintings. https://ieeexplore.ieee.org/document/11386912/

[23] Mapping the chronology of Pecos River style mural painting (Science Advances). https://www.science.org/doi/10.1126/sciadv.adx7205

[24] Intelligent large-scale flue-cured tobacco grading based on deep densely convolutional network. https://www.nature.com/articles/s41598-023-38334-z

[25] A vision transformer model-integrated mobile application for lumpy skin disease detection in cattle. https://pubmed.ncbi.nlm.nih.gov/41318776/

[26] Deep learning for AI-based diagnosis of skin-related neglected tropical diseases: a pilot study. https://pmc.ncbi.nlm.nih.gov/articles/PMC10055440/

[27] Mapping artisanal and small-scale mines at large scale from space with deep learning. https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9498930/